Estado actual del sistema POS/ERP

El Rincón de los Nietos — Informe técnico y funcional

Fecha: 8 de septiembre de 2026

1. Resumen ejecutivo

El repositorio contiene un POS/ERP real construido como monorepo npm workspaces. El frontend es una SPA/PWA con React 19, Vite 7 y TypeScript. El backend usa NestJS 11, Prisma 6, PostgreSQL, JWT, permisos RBAC, Redis tolerante a fallos y Socket.IO para Caja en vivo.

El recorrido más maduro es:

> *Producto → configuración comercial por sucursal → stock → apertura de caja → POS → cobro → venta y pagos → movimiento de stock → ticket → cierre de caja.*

El código compila y tiene cobertura unitaria, pero todavía debe certificarse el recorrido completo mediante HTTP contra una base PostgreSQL de staging con las 23 migraciones aplicadas.

2. Arquitectura

- `apps/admin`: administración, POS, monitor del dueño y administración móvil dentro de una misma SPA.
- `apps/api`: API NestJS modular con guard global de autenticación y permisos.
- `apps/api/prisma`: esquema, seed, sincronización de permisos y migraciones.
- PostgreSQL/API central: fuente autoritativa comercial.
- Redis: infraestructura tolerante a fallos y preparación para funciones en vivo.
- PWA: precachea el shell y assets versionados; no guarda una base comercial offline.
- Producción: SPA bajo `/pos/`, API pública bajo `/pos/api` y API interna en `127.0.0.1:3002/api`.

3. Backend

3.1 Autenticación, usuarios y permisos

El backend ofrece login, refresh, logout y consulta del usuario actual. La sesión incluye empresa, sucursal y permisos. El guard global protege los endpoints y `SUPER_ADMIN` conserva bypass total.

Los permisos cubren panel administrativo y de caja, productos, costos, precios, proveedores, compras, facturas, stock, inventarios, mermas, transferencias, ventas, descuentos, devoluciones, terminales, sesiones de caja, medios de pago, etiquetas, auditoría y Caja en vivo.

3.2 Empresa y sucursales

`Company` representa el tenant. `Branch` representa cada local. La sucursal guarda políticas de margen, impuestos, redondeo, descuentos, precio manual, stock negativo, ticket e impresión. `CompanySetting` y `BranchSetting` permiten persistir configuración JSON sin depender de `localStorage`.

Los recursos se filtran por `companyId`. Stock, compras, terminales, caja y ventas también se limitan por `branchId` y el alcance del usuario.

3.3 Productos

`Product` es el catálogo maestro de empresa. Contiene identidad, códigos, nombre, categoría, subcategoría, marca como atributo, familia, presentación, unidad, contenido neto, condición pesable, precio abierto, imagen, estado y baja lógica.

`BranchProduct` habilita el artículo en una sucursal y conserva costo, precio final, margen, stock mínimo, favorito y reglas comerciales. Un producto puede tener varios `ProductBarcode` y varios proveedores mediante `SupplierProduct`.

La API permite CRUD, baja lógica, duplicación, imagen, importación/exportación, barcodes, configuración por sucursal, historiales, selección server-side y edición masiva con preview y auditoría.

3.4 Motor de precios

`PriceCalculationService` es la autoridad para:

- precio matemático desde costo y margen objetivo;
- margen real y markup real;
- redondeo sin ajuste, a 10, 50, 100, 500, 1.000 o personalizado;
- dirección hacia arriba o al valor más cercano;
- terminaciones comerciales 00, 50, 90 y 99;
- política de cambio de costo `AUTO`, `SUGGEST` o `KEEP`;
- herencia campo por campo: producto, familia, categoría y configuración global.

La fórmula usa margen objetivo, no confunde margen con recargo:

> *precio = costo / (1 - margen / 100)*

Los cálculos críticos usan `Prisma.Decimal`. `PriceHistory` y `CostHistory` conservan valores anteriores/nuevos, origen, usuario y fecha. Un cambio de precio agrega una entrada a `LabelPrintQueue`.

3.5 Stock

`Stock` es el total por producto y sucursal. `StockLocationBalance` separa salón (`SALE_FLOOR`) y depósito (`WAREHOUSE`). `StockMovement` registra tipo, delta, cantidad anterior/nueva, referencia, usuario, motivo y fecha.

`StockService` usa bloqueos transaccionales, valida stock negativo, conserva idempotencia por referencia y genera el movimiento junto con la modificación. Soporta ajuste, stock inicial, reposición depósito→salón, recepción de compras, inventarios, mermas y transferencias.

3.6 Proveedores y compras

Proveedores admite alta, edición, desactivación, detalle y asociación de productos. `SupplierProduct` conserva código, barcode, descripción, último costo, preferencia y datos de bulto. Los aliases ayudan al matching de facturas y a la búsqueda del POS.

Compras incluye órdenes, carga manual, confirmación, cancelación, documentos y análisis asistido con revisión humana.

Confirmar una compra puede actualizar costo, historial, proveedor y precio según las reglas; si cambia el precio genera historial y etiqueta, y la recepción aumenta stock mediante movimientos. La operación crítica es transaccional.

3.7 POS, ventas y pagos

El catálogo POS busca por barcode, código interno, nombre, SKU y datos alternativos/de proveedor. El backend vuelve a validar producto, sucursal, habilitación, precio, permisos, descuento, stock, terminal, caja y pagos al confirmar.

`Sale` conserva `operationId` único, sucursal, terminal, sesión, cajero, totales, costo y ganancia. `SaleItem` guarda snapshots contables. `Payment` conserva medio, importe, impacto efectivo, recibido, vuelto y referencia.

La venta crea venta, líneas, pagos, movimientos de stock y auditoría en una transacción serializable. `operationId` previene duplicaciones. Existen cancelación, devolución y reimpresión sin borrar la venta original.

3.8 Terminales y caja

`Terminal` pertenece a una sucursal y guarda código, nombre, estado, impresora opcional y configuración POS. El código es único por sucursal.

`CashSession` registra terminal, cajero, usuario de apertura, fondo, fecha, cierre, contado y responsable. La base impide dos sesiones abiertas para una misma terminal. `CashMovement` registra ingresos, gastos y retiros con importe, motivo, usuario, origen y fecha.

El efectivo esperado es:

> *fondo inicial + ventas en efectivo + ingresos - gastos - retiros*

El cierre muestra desglose de pagos, esperado, contado y diferencia, y registra auditoría.

3.9 Caja en vivo

Caja en vivo utiliza Socket.IO con autenticación JWT, reconexión y fallback a long polling. `PosLiveEvent` persiste una cronología sanitizada. El monitor puede filtrar por sucursal/terminal y ver sesión, cajero, carrito, total, conexión y actividad. No transmite tokens, referencias de pago, costos ni otros datos sensibles innecesarios.

3.10 Auditoría y etiquetas

`AuditLog` registra operaciones sensibles por empresa, sucursal, usuario, entidad y fecha. `LabelPrintQueue` conserva producto, sucursal, usuario, precio anterior/nuevo, cantidad, estado y fecha de impresión.

4. Frontend

4.1 Aplicación y navegación

La SPA usa un router liviano basado en History API. Intercepta enlaces internos para evitar recargas completas, restaura sesión, aplica permisos por ruta y mantiene redirects de superficies legadas.

El layout tiene header oscuro, verde operativo, sidebar compacto, reloj, usuario, sucursal, conexión, búsqueda global, pantalla completa y menú filtrado por permisos. La administración se organiza en Operación, Ventas y Configuración.

4.2 Dashboard

Muestra ventas de hoy/ayer, variación, tickets, ticket promedio, cajas abiertas, efectivo estimado, medios, alertas de stock y datos comerciales, últimas ventas, resumen por sucursal y accesos rápidos.

4.3 Productos

La pantalla usa búsqueda server-side, debounce, paginación de 20 filas, protección contra respuestas tardías, filtros persistidos por pestaña, cámara para barcode, selección múltiple, importación y editor masivo.

El detalle permite editar información general, configuración comercial por sucursal, stock, proveedores, barcodes, historial y configuración POS. Varias rutas históricas redirigen a Productos, aunque quedan componentes legados que aún se compilan.

4.4 POS

El POS carga medios de pago y bootstrap de caja como dependencias esenciales, y favoritos, grupos y apariencia como configuración opcional. Si las opciones fallan, permite seguir vendiendo.

Incluye apertura rápida, scanner USB, búsqueda, favoritos, grupos táctiles, productos pesables/fraccionados, venta rápida, cantidad, precio autorizado, descuento, nota, eliminación, suspendidos, verificador F8, cobro mixto, vuelto, ticket y cierre. Después de las acciones devuelve el foco al scanner.

El carrito temporal vive en `sessionStorage`. Los suspendidos viven en `localStorage` del dispositivo, por lo que no son compartidos entre cajas ni sustituyen persistencia comercial.

4.5 Administración móvil

Es una superficie responsive dentro de la misma SPA. Permite buscar/escanear, consultar ficha, crear producto, cambiar costo o precio mediante `/pricing/quote`, ajustar stock y enviar etiqueta a la cola. No contiene otro motor de precios.

4.6 Etiquetas

La UI consulta la cola, selecciona pendientes, imprime y marca como procesados. Incluye formatos Fleje, Cartel FyV y A5 Liquidación. La impresión usa HTML/CSS y el diálogo del navegador; no hay spooler nativo ni impresión silenciosa.

5. Estado por módulo

Módulo	Estado real
Autenticación y sesión	Implementado; falta e2e multi-pestaña real
Roles y permisos	Implementado
Multiempresa/multi sucursal	Implementado; debe auditarse continuamente por endpoint
Dashboard	Operativo
Productos	Amplio y operativo
Pricing	Centralizado en backend

Módulo	Estado real
Importación/exportación	Implementada
Edición masiva	Implementada con preview y auditoría
Categorías/familias	Implementadas; integradas en Productos
Proveedores	Operativo
Compras	Backend avanzado; frontend funcional pero mejorable
Facturas asistidas	Parcial; siempre requiere revisión humana
Stock y ubicaciones	Backend sólido; UI operativa
Inventarios/mermas/transferencias	Backend existente; UIs básicas
Vencimientos	Existente; menos maduro
POS	Avanzado y cubierto por tests unitarios
Pesables	Ingreso manual implementado; sin balanza física
Pago mixto y vuelto	Implementados
Venta rápida	Implementada sin stock ficticio
Caja y terminales	Apertura, movimientos y cierre implementados
Ticket	HTML/CSS térmico; sin impresión nativa
Devoluciones	Backend y administración existentes
Caja en vivo	Implementada; depende del proxy Socket.IO
Administración móvil	Implementada en la SPA
Etiquetas	Operativas mediante navegador
Clientes	No existe dominio completo
Cuenta corriente	Sólo medio referenciado; sin mayor de clientes
Reportes avanzados	Incompletos
Alertas programadas	No implementadas
Servidor local/sincronización	No implementados y no simulados

6. Limitaciones y riesgos prioritarios

1. **Certificación de base:** aplicar las 23 migraciones sobre staging y probar APIs reales.
2. **Recorrido integral:** verificar alta, compra, costo/precio, stock, apertura, venta, pago, ticket y cierre.
3. **Clientes:** cuenta corriente no posee cliente, saldo, límite ni mayor contable completo.
4. **Reportes:** falta panel avanzado con filtros, exportaciones y cruces contables.
5. **Suspendidos:** se guardan sólo en el navegador y no se comparten entre terminales.

6. **Código legado:** algunas rutas redirigen correctamente pero sus componentes siguen en el bundle.
7. **Rendimiento:** falta medir con PostgreSQL y 50.000 productos reales.
8. **Caja en vivo:** para varias instancias API necesita un adaptador Socket.IO compartido.
9. **Impresión:** navegador solamente; falta integración nativa con impresora/corte.
10. **Local+nube:** no hay servidor ni PostgreSQL local; la UI no debe mostrar ese estado como real.

7. Conclusión

El sistema posee una base comercial extensa y reutilizable. Productos, pricing, stock, compras, POS, pagos, caja, terminales, etiquetas y auditoría están conectados en código. No corresponde rehacerlos desde cero.

Para declarar la primera versión certificada para los negocios, el próximo paso no es sumar más pantallas: es desplegar en staging, aplicar migraciones, ejecutar el recorrido completo con PostgreSQL real, corregir cualquier inconsistencia y recién entonces completar clientes/cuenta corriente, reportes e impresión nativa en etapas separadas.

8. Fuentes revisadas

- `AGENTS.md`
- `PROJECT_STATUS.md`
- `package.json`
- `apps/admin/src/App.tsx`
- `apps/admin/src/components/Layout.tsx`
- `apps/admin/src/pages/Products.tsx`
- `apps/admin/src/pages/Pos.tsx`
- `apps/api/src/app.module.ts`
- `apps/api/src/permissions/permission-definitions.ts`
- `apps/api/src/modules/pricing/pricing.module.ts`
- `apps/api/src/modules/products/products.module.ts`
- `apps/api/src/modules/purchases/purchases.module.ts`
- `apps/api/src/modules/sales/sales.module.ts`
- `apps/api/src/modules/stock/stock.module.ts`
- `apps/api/prisma/schema.prisma`
- `apps/api/prisma/migrations/`